PLANT DISEASE DETECTION

AND PESTICIDE RECOMMENDATION

A deep-learning image classification project with a Streamlit interface

Project by Gaddam Anish | Team project

01 | Project overview

Plant diseases can reduce crop quality and yield. This project demonstrates an image-based workflow that classifies a submitted leaf image and displays a mapped pesticide recommendation.

Input

A user uploads a leaf image through the web interface.

Prediction

The saved Keras model processes the image and returns class scores.

Result

The app selects the highest-scoring class and displays the detected disease.

Recommendation

A lookup maps the predicted class to a pesticide recommendation.

Project scope: a working prototype for three configured disease classes. It is not a general-purpose diagnostic system for every crop or disease.

02 | Problem & objective

Visual symptoms can be difficult to interpret consistently. The project explores how image classification can support an initial identification workflow using a leaf photograph.

Objectives

- Accept a plant-leaf image from a user.
- Prepare the image in the input format expected by the trained model.
- Run image classification and identify the predicted class.
- Display a disease label and a corresponding recommendation in a simple interface.

What this prototype does not claim

It does not use live weather, IoT sensors, satellite imagery, crop-wide monitoring, or field trials. These topics appeared as background or future possibilities in the original class presentation, not as implemented features.

03 | Application workflow

The implemented flow is deliberately simple: upload → preprocess → predict → map the class → display the result.

1 Upload image	2 Preprocess	3 Model prediction	4 Map class	5 Show result

- **Upload:** the Streamlit file uploader accepts JPG, JPEG, and PNG images.
- **Preprocess:** convert to RGB, resize to 256 × 256, scale pixel values by 1/255, and add a batch dimension.
- **Predict:** call the loaded Keras model and use the highest output index as the predicted class.
- **Display:** show the disease label and its configured pesticide recommendation.

04 | Model & image preprocessing

The application loads a saved Keras model file (plant_disease_model.h5). The project presentation describes the classifier as a Convolutional Neural Network (CNN). The app code shown in the repository performs inference with the saved model; this deck does not claim a specific training procedure or measured accuracy.

Stage	Implementation detail
Image format	RGB colour image
Resize	256 × 256 pixels
Normalize	Pixel array divided by 255.0
Batch	Add a leading batch dimension
Inference	Keras model prediction; select class with argmax

No accuracy percentage, training/validation split, or benchmark result is included because the reviewed project materials do not provide verified metrics.

05 | Configured disease classes

The application maps three output indices to disease names and recommendation text:

Class index	Disease label	Configured recommendation
0	Tomato Bacterial Spot	Copper-based fungicides
1	Potato Early Blight	Mancozeb or Chlorothalonil
2	Corn Common Rust	Azoxystrobin or Propiconazole

Important limitation

Recommendations are hard-coded mappings in the application. They are not generated by the model and should not be treated as agricultural or pesticide-use advice. Users should confirm diagnosis and treatment with a qualified agricultural expert and follow local product labels and regulations.

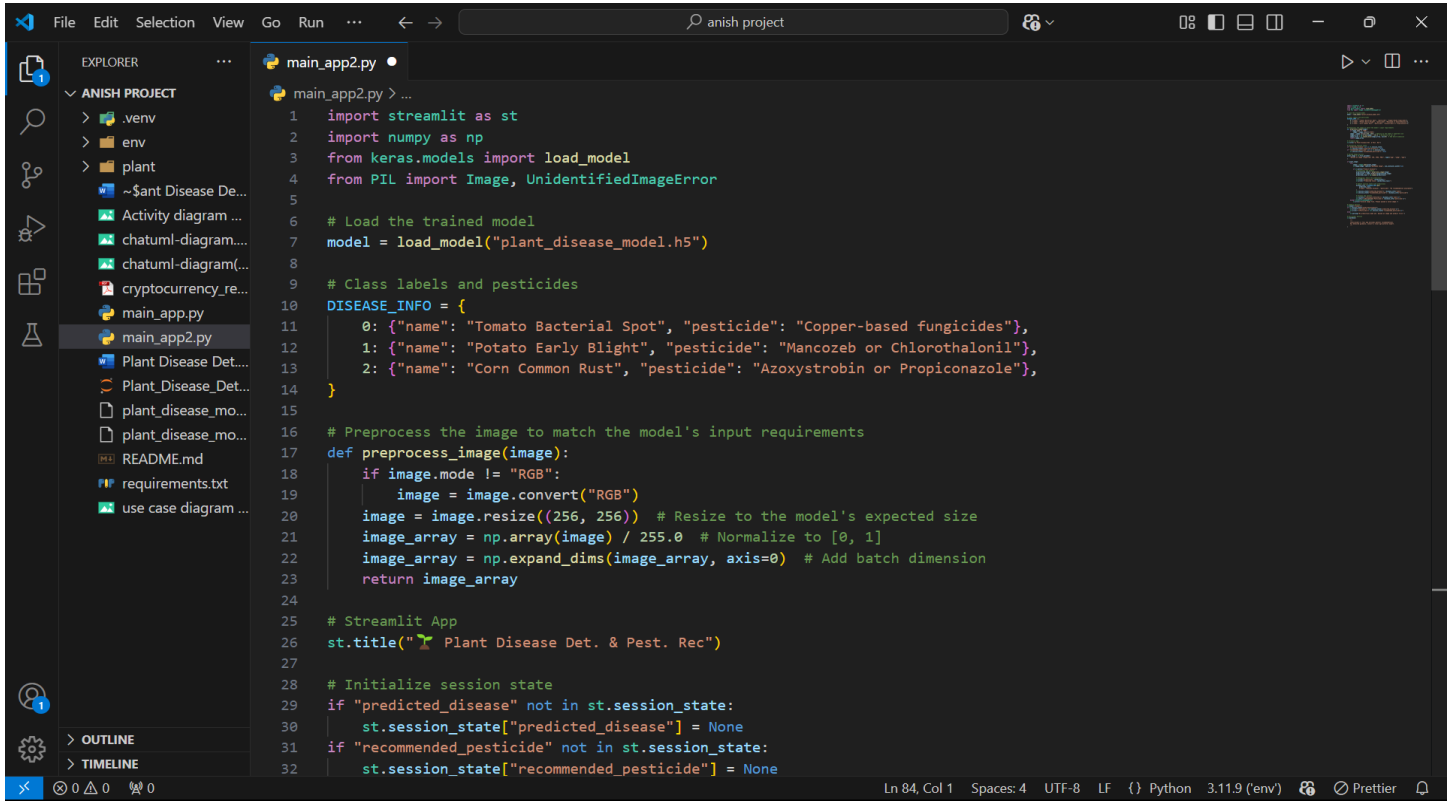
06 | Application architecture

The repository implementation uses a Streamlit interface and a locally loaded Keras model. The original slides mention Flask/FastAPI as possible architecture, but those are not the framework used in the current app.

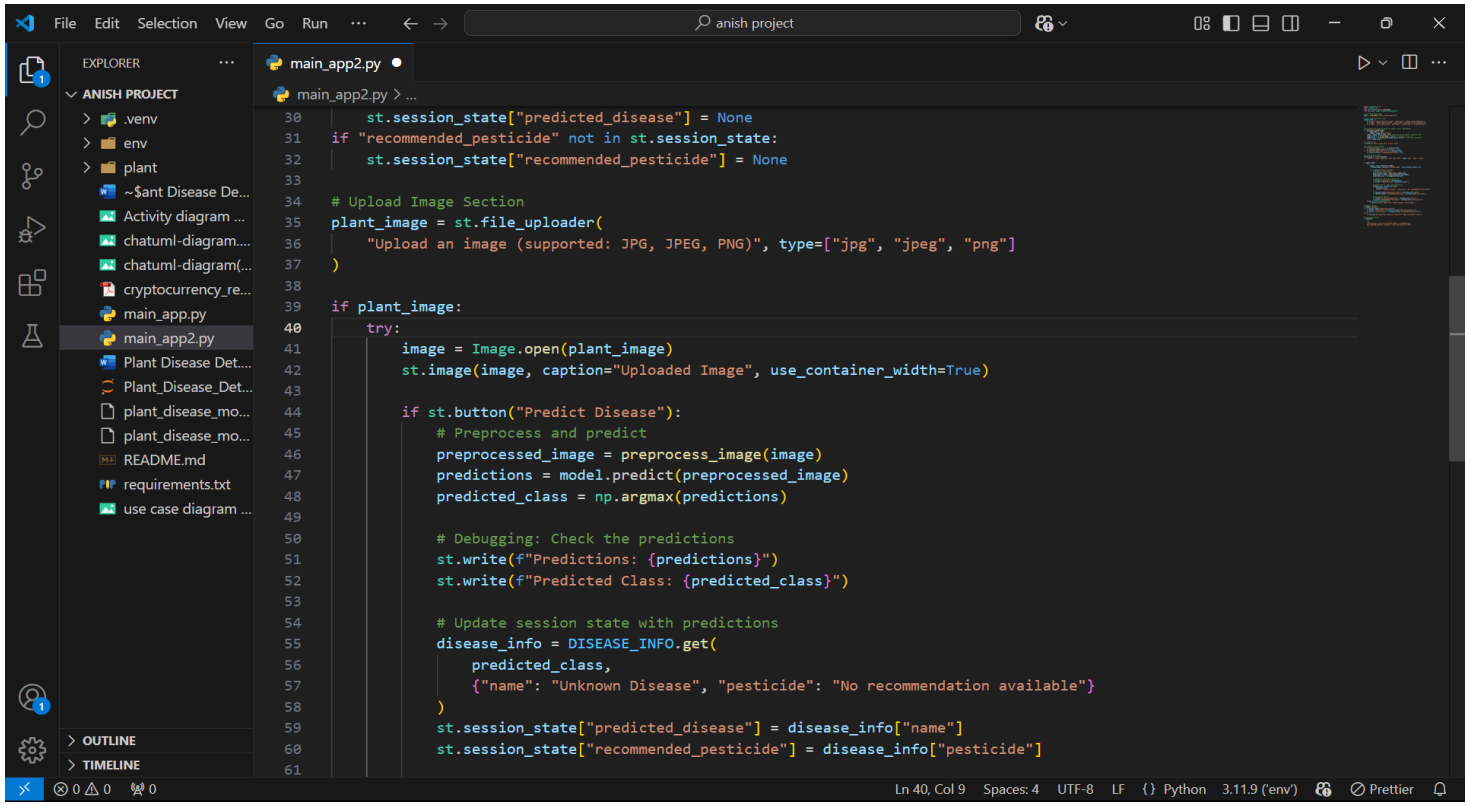
USER
Uploads JPG / JPEG / PNG leaf image
STREAMLIT APPLICATION
Image preview → preprocessing → prediction
KERAS MODEL (.h5)
Predicted class index
DISEASE LOOKUP
Disease name + configured pesticide text

07 | Implementation evidence

The following screenshots are taken from the original project presentation and show the Streamlit application code: model loading and disease mapping, image preprocessing, prediction, and result display.

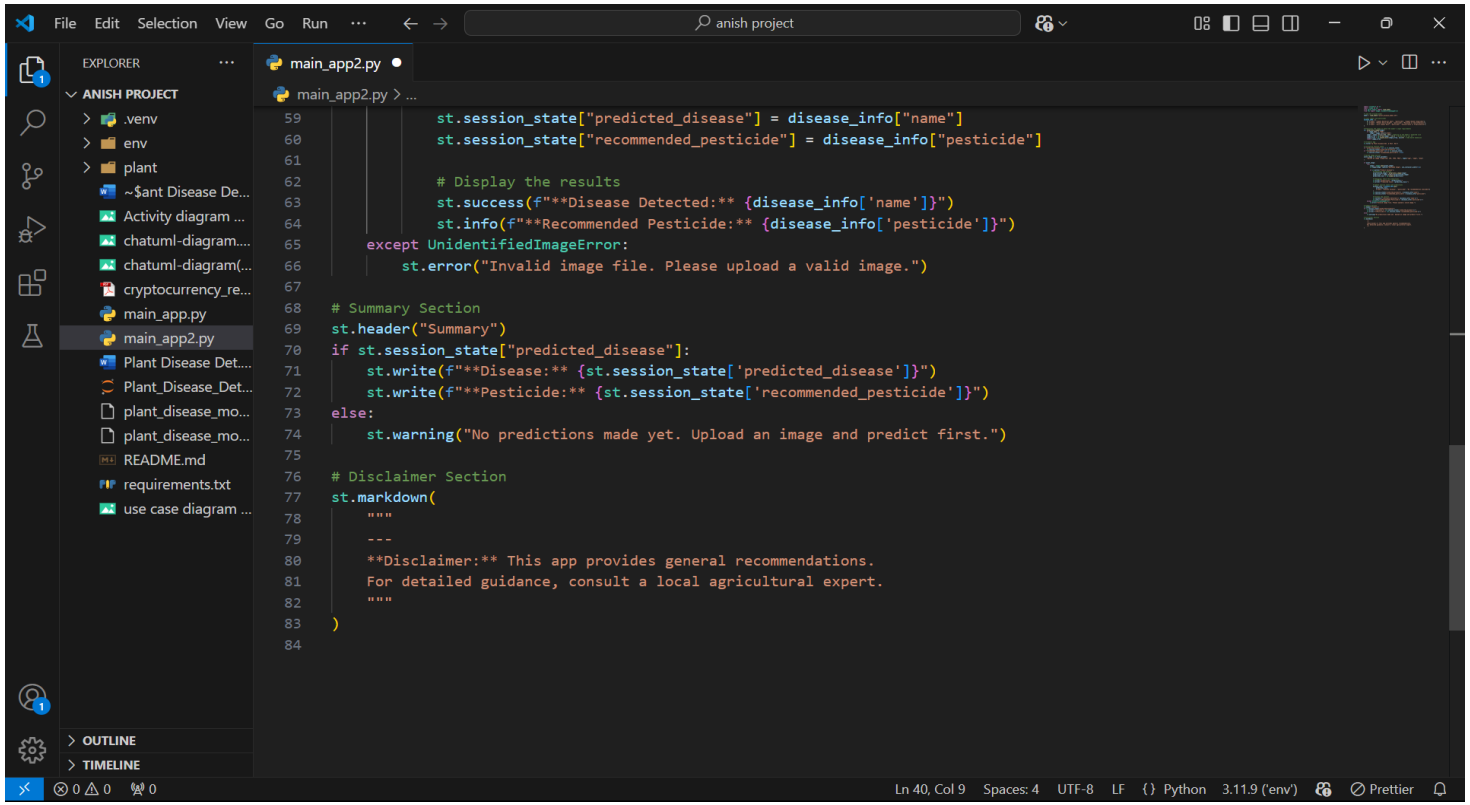


08 | Prediction logic in code



The screenshot shows image loading, prediction invocation, argmax class selection, and lookup of the matching disease information.

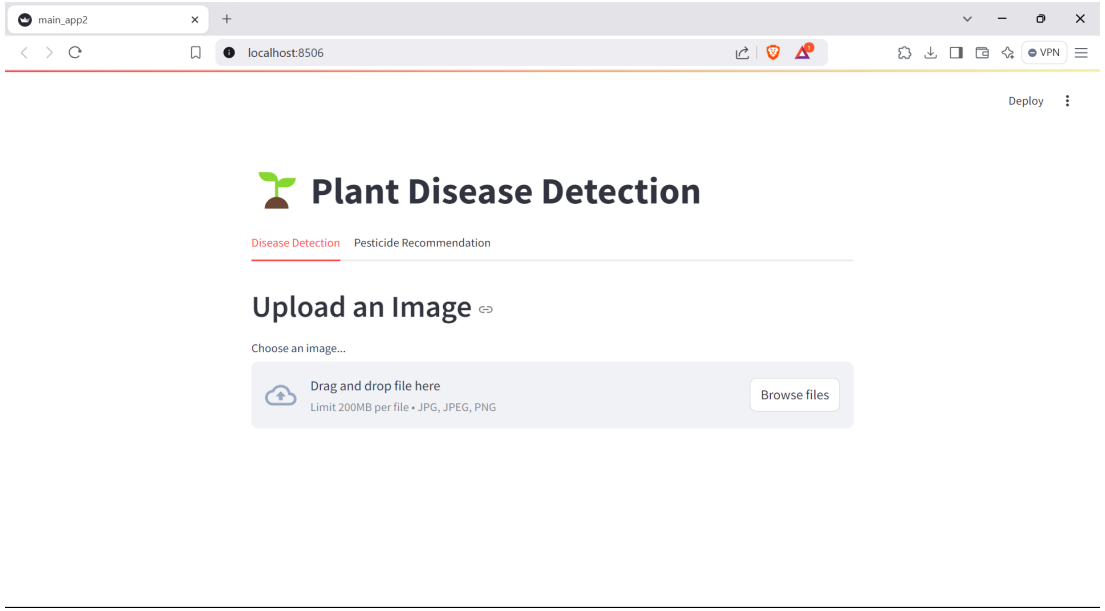
09 | Result display logic

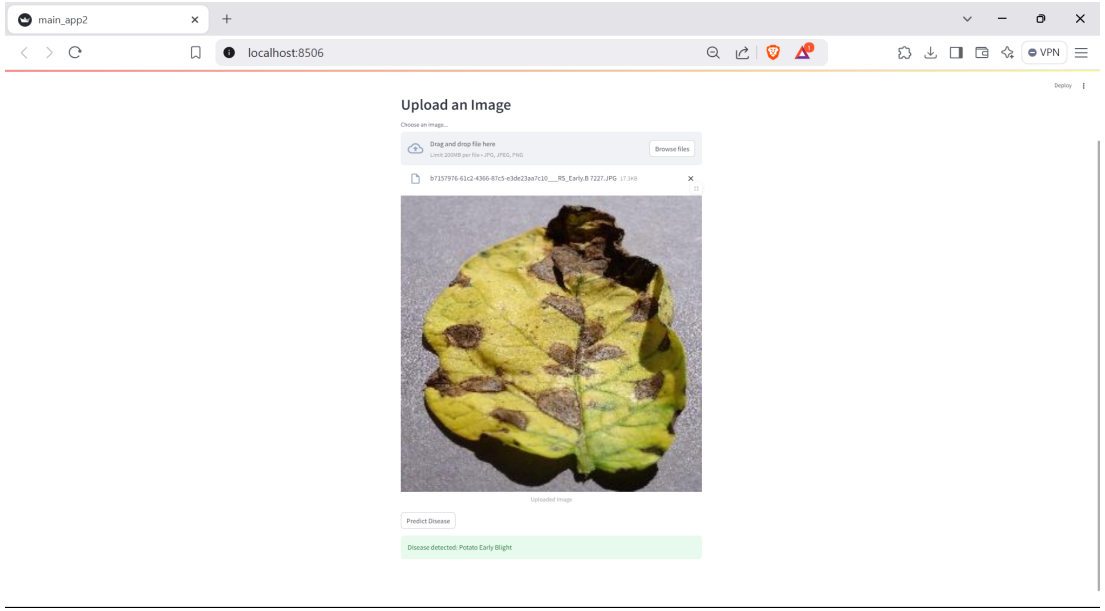


The app displays the predicted disease and recommendation text, with a disclaimer to consult an agricultural expert.

10 | Application screenshots

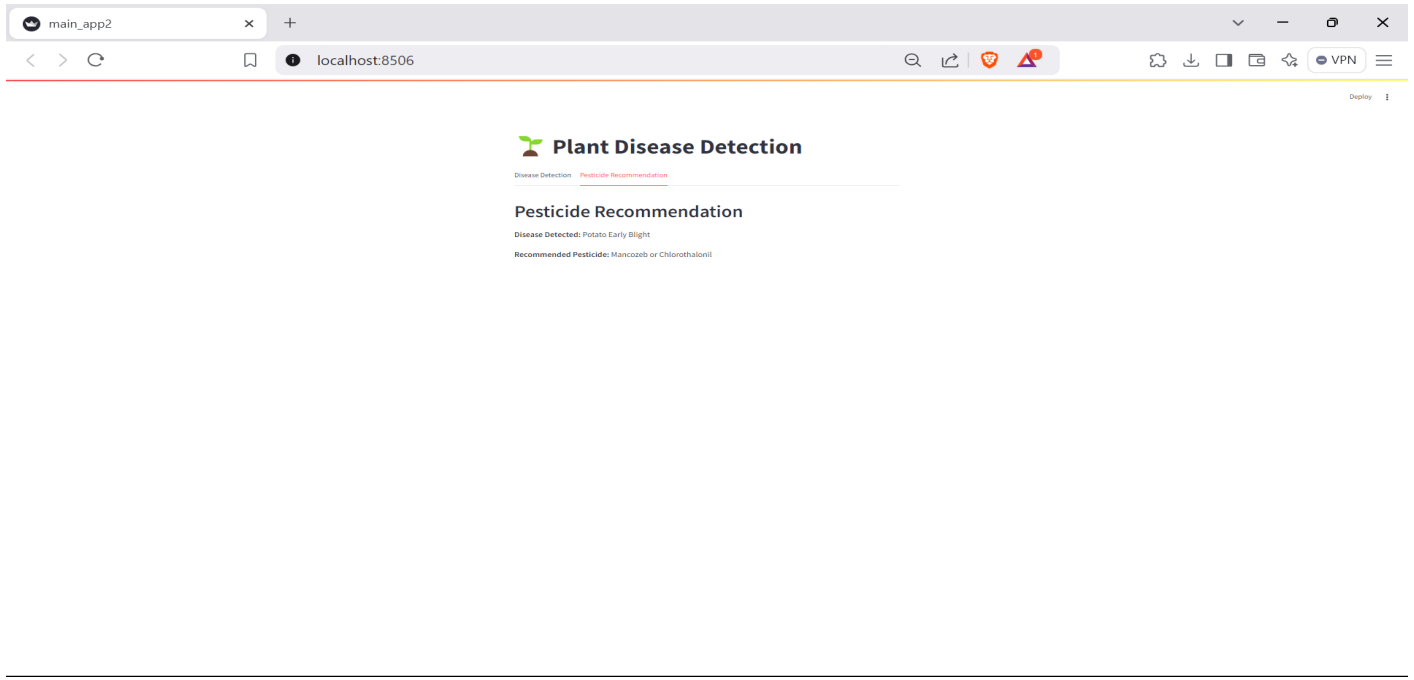
The original presentation includes screenshots of the running Streamlit interface. They document the image-upload screen and a sample prediction flow.





The sample screen shows a leaf image classified as Potato Early Blight. This is an example output, not a published model-accuracy evaluation.

11 | Recommendation interface & technology



Language	Python	**Interface**	Streamlit
ML framework	TensorFlow / Keras	**Image handling**	Pillow, NumPy
Model artifact	plant_disease_model.h5	**Source control**	GitHub

12 | Limitations, next steps & conclusion

Current limitations

• Only three disease classes are configured in the current application.

• The recommendation is a fixed lookup, not a personalized treatment engine.

• The reviewed materials do not provide a verified accuracy metric or external field evaluation.

• The prototype uses uploaded images; it does not integrate live sensors, weather feeds, or remote sensing.

Possible next steps

• Evaluate the model on a clearly separated test set and report per-class metrics.

• Add more disease classes only after validating data quality and model performance.

• Improve input validation and provide a clear low-confidence / uncertain-result path.

• Review pesticide information with qualified agricultural sources and local regulations.

Conclusion

This project demonstrates an end-to-end prototype that combines image upload, image preprocessing, deep-learning inference, disease-label mapping, and a Streamlit user interface. It provides a foundation for further validation and responsible extension.

Repository: https://github.com/gaddamanish06/Plant-Disease-Detection-and-Pesticide-Recomendation